

TLM / TLP 使用手册

TechLib Package (工艺库包) 管理工具 完整使用手册

版本对应：TechLib Package Management Kit V2020_0410a

工具自述版本：TechLib Package (TLP) Management Utility v2020.0410

来源仓库：gitee.com/icdop/tlm

手册日期：2026-09-03

关于本手册的取材说明（请先读这一段）

- 1 你给的链接 <https://gitee.com/icdop/tlm> 目前是需要登录/私有的（git clone 会要求输入用户名，HTTP 返回 401），Gitee 上 icdop 账号公开的 6 个仓库（sta2htm、dop、dvc、tlp、dfa、dqi）中也没有 tlm。
- 2 该项目的公开内容存在于两处，且经逐文件比对完全一致：
 - Gitee 公开仓库：<https://gitee.com/icdop/tlp> (Technology Library Package)
 - GitHub 镜像：<https://github.com/icdop/tlm>
- 3 也就是说：仓库叫 tlm（或 tlp），但里面所有可执行命令的前缀是 tlp_。本手册统一用 TLM 指代这套工具包，用 TLP 指代它所管理的“TechLib Package (工艺库包)”以及 tlp_* 命令。
- 4 本手册不是对 README 的翻译复述。所有命令、输出、错误信息均在 Linux + csh + GNU gawk 5.2 环境下实际运行验证过，包括仓库自带的两个测试用例（run/01_case、run/00_pack）的完整流程。
- 5 源码中若干文档与实现不一致、已声明但未实现之处，已在第 11 章「已知限制与坑」逐条列出。这是本手册最有价值的部分，请务必读完。

目录

- 第 1 章 这套工具是干什么的
- 第 2 章 核心概念与三层数据模型
- 第 3 章 目录结构与代码地图
- 第 4 章 安装与环境配置
- 第 5 章 .tlp 包定义文件格式规范
- 第 6 章 tlp_pack：从设计目录生成 .tlp + .tgz
- 第 7 章 tlp_import：导入包清单到 dataSheets
- 第 8 章 tlp_install：安装工艺库（三种模式）
- 第 9 章 tlp_check 与安装状态追溯文件

- 第 10 章 完整实战：从零搭一套 EDA 工艺库
- 第 11 章 已知限制与坑（实测清单）
- 第 12 章 报错信息速查与排查流程
- 附录 A 命令与选项总表
- 附录 B 可直接复制的 Makefile 与脚本模板
- 附录 C 命名规范与团队落地建议
- 附录 D 术语表

第 1 章 这套工具是干什么的

1.1 一句话概括

TLM (TechLib Package Management Kit) 是一套用 `csh + gawk + tar` 实现的“工艺库 (PDK / IP / 标准单元库 / Memory / GPIO) 版本化打包与安装”工具。它解决的是集成电路设计公司里非常具体的一类运维问题：

晶圆厂 (Foundry) 每隔几周就丢一批 GB 级甚至几十 GB 的 design kit 过来 (PDK 主版本、r1.OHF1...HF7 一连串 hotfix、CTK/ADF/Memory/StdCell/GPIO 等互相有依赖)，要怎么把它打成一个一个带元数据的包、建立一个可检索的“包目录”、让设计工程师只挑自己需要的几个装进自己的 techLib，并且保证任何一台机器上都能按同一份清单，一字不差地重装出一模一样的 techLib？

TLM 给出的答案就是三个命令 + 两种文本文件：

命令	作用	类比
<code>t1p_pack</code>	从设计目录 + CSV 清单 → 生成 .t1p + .tgz	造 rpm/deb 包
<code>t1p_import</code>	把 .t1p 收进 dataSheets/ 并按工艺节点分类归档	建 yum/apt repo 索引
<code>t1p_install</code>	按 .t1p/.dts 描述，把 .tgz 解到 techLib/ 的正确位置	yum install
<code>t1p_check</code>	安装后校验（目前只是空壳）	rpm -V

1.2 它不像、也不是什么东西

- 不是 Python/Go 程序，没有编译产物，没有依赖管理，纯 Shell (csh) 脚本 + awk 脚本，靠符号链接暴露命令。
- 不是 pip/conda。它的“包”是自带一层目录的 .tgz，安装 = `gunzip -c x.tgz | (cd 目标目录; tar xvf -)`，本质是受控的解压 + 打标记文件。
- 没有卸载、没有回滚、没有数据库（状态全是目录里的隐藏文本文件）。
- 只能在 csh/tcsh 下跑（脚本 shebang 是 `/bin/csh -f`，内部大量 csh 专有语法）。

- 硬编码 `/usr/bin/gawk`，很多现代发行版默认只有 `mawk`，会直接找不到。

1.3 版本沿革（从 `doc/ChangeLog.txt` 与 `run/01_case/errors/` 考古）

时间	事件
2017-10-25	建仓，最初是 perl 样例
2017-11-01	定下 <code>sample/ packages/ csh/ run/</code> 四件套结构
2017-11-02	改用 <code>csh/awk</code> ，新增 <code>dcm_install</code>
2017-11-04	为 Cygwin 与 Linux 附带 <code>gawk</code> 可执行文件
2020-04-10	大改名： <code>DCM</code> → <code>TLP</code> ，命令 <code>dcm*</code> → <code>tlp*</code> ； <code>ICFDK</code> → <code>TECHLIB</code> ；工艺节点 <code>1222.2</code> → <code>T28HPC</code> ；版本 <code>x1r0</code> → <code>Op5</code>

这次改名遗留了大量“化石”，是读这套代码时最容易踩的坑：

- `run/01_case/errors/GPIO_lib222_e.0.6.dcm` 还是 2017 年的老 DCM 格式（`DCM FORMAT 1.0 / KIT NODE 1222.2 / REQUIRE KIT ... TOPDIR ... / PACKAGE FILE ... / DCM END`）。现代 `tlp_*` 工具完全不认这套语法。
- `run/01_case/golden/*.log` 全部还是 `dcm_import / dcm_install` 时代的输出，里面的 `releaseNotes/`、`.releaseNote` 后缀也早改成了 `dataSheets/`、`.dts`。这些 `golden` 日志只能当“格式演化的参考”，不能当当前版本的期望输出。
- 因此你在网上/仓库 README 里看到的“老写法”（`GROUP`、`REQUIRE`、`KITNAME`、`DIRNAME`、`MD5 SUM`、`.releaseNote`）大概率是失效的，第 5 章给出的是当前代码真正认的写法。

1.4 什么时候该用 / 不该用

适合：5~50 人的 IC 设计团队，工艺库来源多（多 Foundry、多节点、多 IP vendor），需要“同一套 techLib 在多台机器/多轮片上精确复现”，没有余力自研包管理系统。

不适合：

- 需要真正的依赖求解（TLM 的依赖检查在现版本几乎不生效，见第 11 章）；
- 需要原子性/回滚（它边解 tar 边写，失败会留半成品目录）；
- 需要给非 `csh` 用户（`bash/zsh` 为主）直接使用的场景——需要外面包一层封装。

第 2 章 核心概念与三层数据模型

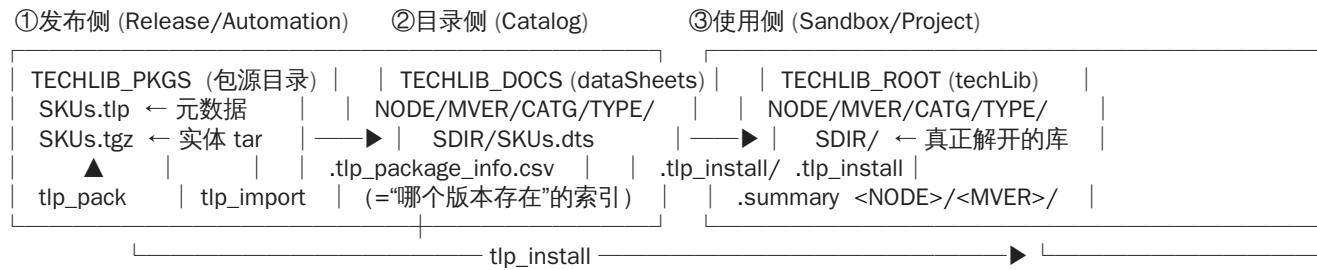
2.1 术语

术语	缩写	含义	例
工艺节点	NODE	制程/平台名	T28HPC、T7FFC、T16FFC
模型版本	MVER	PDK model release 版本	Op5、Op9、Op1

术语	缩写	含义	例
库分组	CATG (不是 GROUP)	techlib 顶层归属	FDK(Foundry Design Kit) / FIP / HIP
库类型	TYPE	设计套件类型	PDK CTK ADF STDCELL MEMORY GPIO DDR SERDES
包名 / SKU	NAME	一个可安装单元的唯一名	PT28HPCPDK_r1.0hf7
源目录	SDIR	包内/安装后的顶层目录名	pdk222_r10HF7
基线版本	ORIG	本版本基于哪个版本 patch 而来	r1.0HF6, 初始版写 _

四层目录顺序恒为：NODE / MVER / CATG / TYPE，其下再挂 SDIR。

2.2 三层数据模型（理解这一张图就会用 TLM）



- .tlp：包定义（元数据 + 装哪个 tgz）。人写的，进版本控制。
- .dts（dataSheet）：.tlp 被 tlp_import 原样拷贝后改了后缀，并放到按 NODE/MVER/CATG/TYPE/SDIR 归类的路径下。内容与 .tlp 一模一样，只是“位置承载了分类信息”。
- .tgz：包实体。约定 tar 内部必须自带一层与 SDIR 同名的顶层目录（实测见 §8.6）。
- techLib/：安装目标。每个库目录下会留一个 <SKU>.dts 作为“已安装”标记，同时在 techLib/.tlp_install/<SKU>.tlp 建符号链接作为“已安装清单”。

2.3 为什么“位置”比“字段”重要

TLM 判断“装没装过”的依据不是版本比较，而是目录里有没有对应的 <SKU>.dts 标记文件；判断“依赖满不满足”的依据是 techLib/.tlp_install/<name>.tlp 这个符号链接是否存在 或 techLib/<req_dir> 路径是否存在。所以：

手工往 techLib 里塞东西 = 骗过 TLM；手工删 techLib/xxx/<SKU>.dts = 让 TLM 认为没装。

这也解释了为什么它没有“卸载”命令：删掉库目录 + 清 techLib/.tlp_install/<SKU>.tlp + 从 <NODE>/<MVER>/<req_dir> 里去掉那行，就是事实上的卸载（见 §9.4）。

2.4 包的两种类型：FULL 与 PATCH

BEGIN TLP <FULL|PATCH> 决定安装时对“目录已存在”的处置，这是 TLM 支持 hotfix 链的机制核心：

情形	FULL	PATCH
目标 SDIR 不存在	正常安装	正常安装（并提示 Base directory ... is missing for patch package.）
目标 SDIR 存在、且目录内有同名 <SKU>.dts	跳过 Skip install (already installed)	同左
目标 SDIR 存在、无同名 .dts	报错冲突：Kit directory ... already exist before installing full kit package.	认可，打印 Directory '...' already exist. 后继续解包

→ 所以一串 hotfix (r1.0.1 → hf1 → hf2 ... hf7) 要写成：首个 FULL，其余全部 PATCH，且 SDIR 全部相同，安装顺序即打补丁顺序。实测见 §10.4。

第 3 章 目录结构与代码地图

```
tlm/ (== gitee.com/icdop/tlp)
├── README.md          # 官方入口文档（部分语法已过期，见 §5.6）
├── Makefile           # 只有一个目标：make install → make bin
├── setup.cshrc        # 一次性设好 TLP_HOME + 3 个 TECHLIB_* 环境变量并打印 tlp_help
├── bin/               # ★命令入口：5 个符号链接，由 make bin 生成
│   ├── tlp_help      -> ../csh/tlp_help.csh
│   ├── tlp_pack      -> ../csh/tlp_pack.csh
│   ├── tlp_import    -> ../csh/tlp_import.csh
│   ├── tlp_install   -> ../csh/tlp_install.csh
│   └── tlp_check     -> ../csh/tlp_check.csh
├── csh/               # ★全部实现（约 800 行）
│   ├── tlp_header.csh (45 行) # 打印蓝色 banner、开/轮转日志、吃掉 --log
│   ├── tlp_option.csh (149 行) # ★统一的选项解析器 + 环境变量默认值
│   ├── tlp_help.csh  (60 行) # 内建帮助/dispatch (readme/format/env/command/run...)
│   ├── tlp_import.csh (59 行) + tlp_import.awk (193 行) # 建 catalog
│   ├── tlp_install.csh (183 行) + tlp_install.awk (278 行) # ★安装引擎 + 交互菜单
│   ├── tlp_pack.csh  (38 行) + tlp_pack.awk (224 行) # 打包
│   └── tlp_check.csh (30 行) # ★空壳，未实现
├── doc/
│   ├── TLP_FORMAT.md # ★.tlp 语法正式文档（与代码最接近）
│   ├── TLP_SPEC.txt  # 早期安装流程草稿笔记
│   ├── COMMAND.md    # 命令文档“模板占位”，内容其实是空的
│   └── ChangeLog.txt  # 版本沿革（DCM → TLP 改名记录）
├── etc/make/bin.make  # 生成 bin/ 软链的 make 片段
├── run/
│   ├── 00_pack/       # 实战用例 A：CSV 清单 → 打包 → 建 bundle → 复现安装
│   │   ├── designkit_r061.csv / designkit_r101.csv / patch_pdk222_r101.csv
│   │   ├── pdk222_r10hf.csv # ★遗留文件，列顺序与代码不匹配（见 §11-9）
│   │   ├── designkit/ patch/ designkit_patch/ # 假的设计套件源目录
│   │   └── Makefile # 目标：help / pack / patch / import / install / bundle / diff / clean
│   ├── 01_case/      # 实战用例 B：已有 .tlp+.tgz → import → 交互安装 → bundle 复现 → diff
│   │   ├── configs/*.tlp (10 个) packages/*.tgz (15 个) script/install_test.cmd
│   │   ├── errors/GPIO_lib222_e.0.6.dcm # ★2017 老 DCM 格式标本
│   │   ├── golden/ # ★2017 时代 dcm_* 输出留档
│   │   └── Makefile # 目标：env/import/error/install/install_test/bundle/summary/diff/clean
```

3.1 读码顺序建议

1 csh/tlp_option.csh —— 15 分钟读完，就掌握了全部命令行与默认值。

- 2 `doc/TLP_FORMAT.md` —— `.tlp` 语法。
- 3 `cshtlp_install.awk` 的 `ENDFILE` 块（第 192–255 行）——全部安装语义、冲突判定、状态文件写入都在这 60 行里。
- 4 `cshtlp_install.csh` 第 81–179 行 —— 交互菜单。
- 5 `cshtlp_pack.awk` 第 105–211 行 —— CSV 到 `.tlp/.tgz` 的映射。

第 4 章 安装与环境配置

4.1 依赖清单

依赖	用途	是否硬性	说明
<code>csht</code> (或 <code>tcsh</code>)	解释所有 <code>.csh</code>	必须	shebang 写死 <code>/bin/csh</code> <code>-f</code> ; <code>bash/zsh</code> 下无法直接跑
<code>gawk</code>	解释所有 <code>.awk</code>	必须	<code>import/install/pack</code> 里写死绝对路径 <code>/usr/bin/gawk</code> ; 且用了 <code>BEGINFILE/ENDFILE</code> (<code>mawk</code> 不支持) , 也写了 <code>/usr/bin/awk</code> 的分支 (见 §11-5)
<code>tar</code> / <code>gzip</code>	解包打包	必须	打包用了老式 <code>tar -c -O -z</code> (见 §11-13)
<code>tree</code>	交互菜单画目录树	必须	需要支持 <code>-C -d -L 4</code> 、 <code>--noreport</code>
<code>md5sum</code>	—	形式上需要	代码里算了但从不校验 (见 §11-3)
<code>find/glob/ls/diff/tee/xpdf</code>	辅助	大部分必须	<code>tlp_help start</code> 会调 <code>xpdf</code> (且指向不存在的 <code>docs/</code> 目录, 见 §11-14)

Debian/Ubuntu 一次性装齐：

```
sudo apt-get install -y csh gawk tree
```

4.2 部署

```
# 1) 取代码 (Gitee 的 tlm 需登录; 下面两个公开源内容一致)
git clone https://gitee.com/icdop/tlp.git tlm
# 或 : git clone https://github.com/icdop/tlm.git tlm
```

```
# 2) 生成 bin/ 软链 (唯一需要“编译”的步骤)
cd tlm
make install      # 内部只是 make bin
# 验证 : ls -l bin/ 应看到 5 个指向 ../csh/*.csh 的符号链接
```

```
# 3) 挂到 PATH
```

```
setenv TLP_HOME `pwd`
set path = ($TLP_HOME/bin $path)
```

Makefile 的 install 目标只建软链，不会装到 /usr/local。要全局可用，请自行把 \$TLP_HOME/bin 加入 PATH（或整目录 cp -r 到共享路径后同样加 PATH）。

4.3 环境变量

变量	命令行选项	含义	代码内默认值
TLP_HOME	—	工具根目录；不设则由脚本用 \$0:h/.. 自行推导	自动推导
TECHLIB_PKGS	-p / --packageSrcDir	包源目录（放 .tgz），可被 tlp_import 当 .tlp 源	packages
TECHLIB_CFGS	-c / --packageCfgDir	.tlp 定义文件目录	\$TECHLIB_PKGS
TECHLIB_DOCS	-r / --dataSheetDir	dataSheet 目录（catalog）	dataSheets
TECHLIB_ROOT	-t / --targetLibDir	安装目标 techLib	techLib
TECHLIB_TEMP	--tempDir	打包暂存区	tempLib
TLP_CFGS_DEST	--packCfgDir / --packCfgsDir	tlp_pack 输出 .tlp 的目录	\$TLP_PKGS_DEST
TLP_PKGS_DEST	--packDestDir	tlp_pack 输出 .tgz 的目录	packages
TECHLIB_OPTION	—（环境变量专用）	传给 awk 的 --verbose/--info/--test	空

三个要点：

- 1 TECHLIB_PKGS 支持冒号分隔多路径（tlp_import.awk/tlp_install.awk 内 split(..., ":") 逐个试找 tgz）。这可以用来做多源（本地缓存 + 网络盘）：

```
setenv TECHLIB_PKGS "/data/tlppkg/local:/data/tlppkg/foundry_A:/data/tlppkg/vendor_ip"
```
- 2 TECHLIB_CFGS 默认等于 TECHLIB_PKGS。若你把 .tlp 和 .tgz 分家存放（推荐的 01_case 式布局：configs/ + packages/），必须显式给 --packageCfgDir，否则按 SKU 名安装会找不到定义文件。
- 3 默认值全是相对路径（techLib、dataSheets、packages）。所有命令都在当前工作目录下干活，跨目录调用会各自生成一套 techLib/。团队落地时建议统一给绝对路径（附录 B 模板已这么做）。
- 4 官方 setup.cshrc 给的定义是：TECHLIB_PKGS=\$TLP_HOME/packages、TECHLIB_DOCS=dataSheets、TECHLIB_ROOT=targetLib。注意 \$TLP_HOME/packages 这个目录仓库里并不存在（.gitignore 也没管），照抄会报 “No match”。

4.4 通用命令行骨架（★顺序有讲究，实测规则）

```
tlp_<cmd> [选项...] [位置参数...]
```

- 所有选项必须写在位置参数前面。解析器（tlp_option.csh）一遇到第一个非选项 token 就停止，之后所有 token 都被当成“包名”。


```

tlp_install --targetLibDir t1_ok PT28HPCPDK_r0.6.1      #
tlp_install PT28HPCPDK_r0.6.1 --targetLibDir t1_ok      # ERROR: 找不到 ...'--targetLibDir'

```

- `--log <file>` 必须是第一个选项。它由 `tlp_header.csh` 消化，而该 case 缺 `breaksw` 会顺带把解析停下；一旦 `--log` 出现在别的选项后面，`tlp_option.csh` 不认识它就把它当包名。

```

tlp_install --log run1.log --targetLibDir I1 PT28HPCPDK_r0.6.1 # 实测成立
tlp_install --targetLibDir I2 --log run2.log PT28HPCPDK_r0.6.1 # ERROR: Can not find TLP config '--log'

```

- 每个 `tlp_*` 都带 `-h/--help` 打印自己的用法（内容与本手册附录 A 一致，但不含 `--packageCfgDir` 之外的全部选项，`tlp_install --help` 就没列 `--packageCfgDir`，实际可用）。
- 日志文件自动轮转：已有 `tlp_import.log` 时再次运行会依次变成 `.log.1 .log.2 ...`（不会覆盖历史）。

4.5 自检

```

tlp_import -i --packageCfgDir configs |& grep TECHLIB
# 期望看到四行回显当前 ROOT/DOCS/PKGS/CFGs
# ★注意：这里的 "TECHLIB_CFGS = ..." 打印的是 $TECHLIB_PKGS 的值（源码 bug，见 §11-11）

```

第 5 章 .tlp 包定义文件格式规范

编写依据：`csh/tlp_import.awk`、`csh/tlp_install.awk`、`csh/tlp_pack.awk` 里实际生效的正则，而不是 README 的示例。

5.1 词法约定（先记住这 5 条）

- 1 字段以 空白（Tab 或空格）分隔，仓库内文件一律用 Tab 对齐；解析用 `awk` 默认 `$N`，所以列数（第几字段）就是语义，多一个少一个都会串位。
- 2 关键字必须顶格、大写、且后面必须跟至少一个空白（正则 `/^NAME\s/` 这种，`NAME` 后没空格则整行被忽略）。
- 3 `#` 开头整行跳过（`/^#/`）。
- 4 `;` 不是注释！它只是不匹配任何关键字正则所以看起来无害——但 `;PATCH T28HPC ...` 这种行恰好不匹配，而 `;` 开头的行会被 `tlp_pack` 的 `BEGIN PACKAGE` 之类的解析绕开。规范里把 `;` 当注释是历史习惯（如 `00_pack/*.csv`），建议只用 `#` 注释。
- 5 遇到 `END` 关键字（正则 `/^END\s/`，需要 `END` 后至少跟一个空白 `**`）`**` 才停止解析该文件，`END` 之后可以写任意人类可读说明。
实测：光写一行 `END`（行尾没有任何空格/字符）不会截断解析，后面的内容仍会被读，并且按 `awk` 的“后写覆盖”规则改写前面的分类字段：

```

# 结尾写裸 END，其后还有 SDIR sdir_after_end / NODE BBB
$ tlp_import --packageCfgDir e3
: Creating 'BBB/1p0/FDK/PDK/sdir_after_end/ENDTEST_bare.dts' ← 被 END 之后的行改掉了

```

仓库自带的 `.tlp` 全部以裸 `END` 结尾，只是因为后面确实没有更多关键字才无害。规范写法：`END` 结尾留一个空格，或 `END`；说明文字。

5.2 五个区块与全部关键字


```

BEGIN TLP <FULL|PATCH> ← ①头：包类型
NAME <SKU> ← ②标识
ORIG <基线版本|_|->
NODE <工艺节点> ← ③分类（决定安装路径）
MVER <模型版本>
CATG <库分组> ★必须是 CATG，不是 GROUP
TYPE <库类型>
SDIR <顶层目录名> ← ④内容与完整性
SIZE <KB，仅记录>
MD5S <tgz 或目录的 md5，仅记录>
TDIR <目标目录，代码未使用>
DEPN KIT T28HPC/0p5/FDK/PDK TOPDIR pdk28_r061 ← ⑤依赖（★推荐写法）
DEPN KIT = <依赖包名> ← 名称式依赖：`KIT=` 与名字之间必须有空白
DEPN DIR = <依赖目录> ← 路径式依赖：同上
REQU <基线包名> <基线包名> ← ⑤依赖与载荷（★同名写两遍，见 §5.3/§11-6）
FILE <xxx.tgz> [<topdir>] [<md5>]
END

```

5.3 逐字段说明（含“代码到底怎么用这一列”）

关键字	字段	是否必需	实际作用（已按代码核实）
BEGIN	\$2=TLP, \$3=类型	必需	\$3 为时空默认 FULL。FULL/PATCH 决定 §2.4 的目录冲突策略。其余值（如写成 MULTI）会被原样记录但不影响逻辑。
NAME	\$2	建议	仅作展示（日志里 Creating '...' (NAME)）。安装时真正的包名是从文件名推导的（basename .dts 再去 .t1p），所以 NAME 与文件名不一致不会报错，只会让 bundle/复现时迷惑。→ 见 §5.5 铁律。
ORIG	\$2	建议	记录基线版本；无基线写 _。仅记录，不做比较。
NODE	\$2	必需	决定 dataSheets/ 与 techLib/ 的第一层；缺了它 → 路径出现空段（§11-2）。
MVER	\$2	必需	第二层。
CATG	\$2	必需	第三层（FDK/FIP/HIP）。写成 GROUP 会被完全忽略。

关键字	字段	是否必需	实际作用（已按代码核实）
TYPE	\$2	必需	第四层。TYPE 一行两用：在 BEGIN 行里 \$2 是包类型，在独立行 / [^] TYPE\s/ 是库类型；因为 BEGIN 行由 / [^] BEGIN\s+TLP\s/ 优先匹配，不冲突。
SDIR	\$2	必需	安装第五层目录名 + “已安装”判定目标 + 与 tgz 顶层目录必须一致 (§8.6)。
SIZE	\$2	可选	纯记录（awk 里赋值后仅在 --verbose 时打印）。
MD5S	\$2	可选	纯记录，从不校验 (§11-3)。
TDIR	\$3	勿用	赋给 tlp_package_dir 后再无引用（死变量）。目标目录一律由 SDIR + tgz 内部结构决定。
DEPN KIT <类别路径> TOPDIR <topdir>	\$3,\$5	可选	检查 techLib/<类别路径>/<topdir> 是否存在。★唯一“照字面写就对”的形式，推荐用它。2017 年老 REQUIRE 的继任者。
DEPN KIT= <依赖SKU> (KIT= 后必须空白)	\$3	可选	检查 techLib/.tlp_install/<依赖SKU>.tlp。_/- 视为无依赖。
DEPN DIR= <依赖目录> (同上需空白)	\$3	可选	检查 techLib/<依赖目录>。
DEPN KIT=<name> (等号后无空白)	—	请勿	★死路：\$2 吃走整串、\$3 为空 → 恒定判定“依赖缺失”，永远装不上（实测，报 Required kit '' has not been installed yet.）
DEPN <包名> (两个字段)	—	样例里常见	★不匹配任何规则，等于没写！（§11-4）仓库自带 configs/*.tlp 与 tlp_pack 默认产出全用这种写法 → “依赖检查”其实是空转的。

关键字	字段	是否必需	实际作用（已按代码核实）
REQU	\$2 (import) /\$3 (install)	可选	TLM 唯一会自动装基线的机制：install 遇到 REQU 会自己 system("tlp_install <CFGs>/<基线>.tlp") 先补一层基线（要求 tlp_install 在 PATH 上）。 两个读者列位不同 → 基线名必须写两遍 REQU <基线SKU> <基线SKU>；只写一列时 install 读到空字段，该包直接 (missing package) 装不上（实测）。递归只补一层，且不看 bundle 顺序。
FILE	\$2=tgz 名，\$3=可选 topdir，\$4/\$5=可选 md5	必需（至少一行）	载荷清单。多行 = 分包（大包拆 -1.tgz -2.tgz …），按出现顺序依次解。查找范围是把 \$2 在 TECHLIB_PKGS 的冒号路径列表里逐个试。PATCH 关键字也会被同一规则接收（老写法兼容）。
END	—	必需	终止解析。

tlp_pack 生成的 .tlp 只会写 BEGIN/NAME/ORIG/NODE/MVER/CATG/TYPE/SDIR + 至多两条 DEPN + 一条 FILE + END（见 §6.4）。手写时可用的字段是它的超集。

DEPN 五种写法实测矩阵

同一场景（ADF 依赖 PDK；依赖未装 / 已装两种前置状态）下逐一验证：

写法	依赖未装时	依赖已装时	判定
DEPN KIT <NODE/MVER/CATG/TYPE> TOPDIR <topdir>	(dependency fail)	Total 1/1 kits are installed.	推荐
DEPN KIT= <SKU>（等号后带空白）	(dependency fail)	Total 1/1 kits are installed.	有效但反直觉
DEPN DIR= <NODE/MVER/CATG/TYPE>/<topdir>（同上）	(dependency fail)	Total 1/1 kits are installed.	有效但反直觉
DEPN KIT=<SKU>（等号后无空白）	(dependency fail)	仍然 (dependency fail)	死路（\$3 为空，恒不满足）

写法	依赖未装时	依赖已装时	判定
DEPN <SKU> (只有两列)	Total 1/1 kits are installed.	—	完全不检查 (自带样例 即此种)

结论：要么老实用 DEPN KIT <类别> TOPDIR <topdir>，要么记住 KIT= 后面必须再跟空白。
两种“最像正常配置”的写法都是错的。

5.4 两份可直接抄的完整样例

A. 标准全量包 (FULL，一个 tgz)

```
BEGINTLPFULL

NAMEGPIO_lib222_e.0.6.1
ORIG_
NODET28HPC
MVEROp5
CATGHIP
TYPEGPIO
SDIRip222_gpio_r061
SIZE10000
MD5S11ba9bfa12c16459bc242c005c351b6f

DEPNKIT28HPC/Op5/FDK/PDKTOPDIRpdk222_r061

FILEGPIO_lib222_e.0.6.1.tgz

END
; 以下为人工说明，工具忽略。
; 变更：e.0.6.1 修正 GPIO upf 端口名。
```

B. hotfix 包 (PATCH，依赖目录 + 多载荷 tgz)

```
BEGINTLPPATCH

NAMEPT28HPCPDK_r1.0hf7
ORIGr1.0HF6
NODET28HPC
MVEROp5
CATGFDK
TYPEPDK
SDIRpdk222_r101
SIZE10000

DEPNKIT T28HPC/Op5/FDK/PDK TOPDIR pdk222_r10HF4

FILEPT28HPCPDK_r1.0hf5.tgz
FILEPT28HPCPDK_r1.0hf6.tgz
FILEPT28HPCPDK_r1.0hf7.tgz

END
```

C. 单个 .tlp 覆盖多分卷 (超大库拆分)

```
BEGINTLPFULL
NAMESTDCELL_lib222_7t_base_e.2.0
ORIG7t_base_e.1.1
NODET28HPC
MVEROp5
CATGFIP
TYPESTDCELL
```

```
SDIRlib222_7t_base_e20
SIZE30000

FILESTDCELL_lib222_7t_base_e.2.0-1.tgz
FILESTDCELL_lib222_7t_base_e.2.0-2.tgz
FILESTDCELL_lib222_7t_base_e.2.0-3.tgz
```

END

分卷的正确做法是：每卷只含 SDIR/ 的一部分文件，按顺序解到同一父目录，tar 会自然合并（01_case/packages/STDCELL_lib222_7t_base_e.2.0-{1,2,3}.tgz 即如此）。若两卷含同名文件，后解的覆盖先解的。

5.5 三条铁律（违反必翻车）

- 1 文件名 == NAME == 主 FILE 去掉 .tgz == bundle 清单里的那一行。TLM 到处用文件名推 SKU（.tlp → .dts → <SKU>.dts 标记 → .tlp_install/<SKU>.tlp → .tlp_packages 行）。任何一处不一致，交互菜单、复现安装、依赖符号链接就会指向不同名字。
- 2 SDIR == tgz 解压出来的顶层目录名。不等就会出现“标记文件建在 A 目录、内容解在 B 目录”，第二次装同一族包时报 conflict root（实测：自带 PT28HPCPDK_r1.0hf7.tlp 的 SDIR=pdk222_r101 而 tgz 顶层是 pdk222_r10HF7）。
- 3 一个 SDIR 只能被一个 FULL 族拥有。hotfix 链上除第一个是 FULL，其余必须 PATCH，且 SDIR 保持一致。

5.6 README 示例 vs 实际可用写法（踩坑对照表）

README.md 里的写法	现代 tlp_* 是否认	正确写法
GROUP FIP	忽略	CATG FIP
TYPE STDCELL（在 BEGIN 下第一段）		同
KITNAME 7t_base_e.2.0	忽略	NAME <SKU>
ORIGIN ...	忽略	ORIG ...
DIRNAME lib222_7t_base_e20	忽略	SDIR ...
MD5SUM ...	忽略	MD5S ...
PACKAGE FILE xxx.tgz	半可：/^PACKAGE\s/ 不匹配，但下一列会被当噪声	FILE xxx.tgz
REQUIRE KIT <cat> TOPDIR <top>	仅 tlp_import 认（且取 \$2="KIT"）	DEPN KIT <cat> TOPDIR <top>
BEGIN TLP（无类型）	默认 FULL	BEGIN TLP FULL
TECHLIB_PKGS - package source directory (*.tgz and *.tlp)		建议仍分家为 CFGS/PKGS
交互示例里的 INFO: Validating package integrity ... / Checking package dependcy ...	代码里没有这两步输出	实际只有 Package file - / Unpacking file

README.md 里的写法	现代 tlp_* 是否认	正确写法
\$TECHLIB_ROOT/.tlp_install.summary	拼写错且路径不完全对	实为 techLib/.tlp_install.summary

第 6 章 tlp_pack : 从设计目录生成 .tlp + .tgz

tlp_pack 是发布侧工具：给它一份 CSV 清单，它为每个 kit 生成 .tlp（写进 TLP_CFGS_DEST）与 .tgz（写进 TLP_PKGS_DEST），并把 SKU 追加到 <csv名>.bundle。

6.1 用法

```
tlp_pack [--packCfgsDir <出.tlp目录>] [--packDestDir <出.tgz目录>] [--tempDir <暂存>] <list.csv> ...
```

run/00_pack/Makefile 里的真实调用：

```
set env TLP_PACK_OPT = --packCfgsDir configs --packDestDir packages --tempDir tempLib
tlp_pack $TLP_PACK_OPT designkit_r061.csv designkit_r101.csv # 全量 collateral
tlp_pack $TLP_PACK_OPT patch_pdk222_r101.csv # hotfix 链
```

6.2 CSV (PACKAGE) 格式

```
BEGINPACKAGE1.0      ← 版本号字段目前只是记录

NODET28HPC           ← 默认分类，可被每行的显式列覆盖
MVEROp5
DDIR<默认 topdir>    ← 代码未实现，仅占位

#-----
;PACK NODE MVER CATG TYPE NAME SDIR VERSION REQU LOCATION CONTENT
FULL T28HPC Op5 FDK PDK PT28HPCPDK_r0.6.1 pdk222_r061 r0.6.1 - designkit/pdk222_r061
PATCH T28HPC Op5 FDK PDK PT28HPCPDK_r1.0hf1 pdk222_r101 r1.0HF1 PT28HPCPDK_r1.0.1 designkit_patch/

END
```

列位 (awk 严格按 \$1…\$11 取)：\$1=PACK 类型 (FULL|PATCH|MULTI)，\$2=NODE，\$3=MVER，\$4=CATG，\$5=TYPE，\$6=NAME(SKU)，\$7=SDIR，\$8=VERSION，\$9=REQU (基线 SKU)，\$10=LOCATION (源目录或已有 tgz)，\$11=CONTENT (LOCATION 内的打包范围，默认 .)。

表头必须用 ; 或 # 起头 (/^#/ {} 会跳过 # ; ; 行不匹配任何规则，等效跳过)。run/00_pack/designkit_r061.csv 就是拿 #----- 画横线 + ;PACK … 写表头的写法。

6.3 LOCATION (第 10 列) 的四种语义

LOCATION 写法	行为
空 / - / _	只生成 .tlp，不生成 .tgz (Create TLP file only …) ——用于引用外部已有包
一个 已有 .tgz 文件路径	直接引用它 (Use package file '…')
一个 目录，且 basename == SDIR	就地 (cd 父目录; tar -c -O -z SDIR) > 包名.tgz

LOCATION 写法	行为
一个 目录，且 basename ≠ SDIR	先 (cd LOCATION; tar -c -O CONTENT) (cd tempLib/.../SDIR; tar -x) 归一化目录名，再从 tempLib 打 tar
不存在	ERROR: Kit directory '...' does not exist. 并 rm -rf 暂存目录，计入 error

MODE TEST 行（或 ;TLP MODE TEST 取消注释）会让 pack 进入演练模式：只造一个假目录 + 包名_test.tgz，用于快速验证清单语法。

\$1 == MULTI 时不打包，而是 ls -l LOCATION/*.tgz 把该目录下所有 tgz 列成多行 FILE —— 对应“vendor 已经给了我一批分卷”的场景。

6.4 生成结果长什么样（实测）

designkit/ctk222_r061 一行 FULL 生成的 configs/PT28HPCADF_r0.6.1.tlp：

```
BEGIN TLP FULL

NAME PT28HPCADF_r0.6.1
ORIG_
NODE PT28HPC
MVER Op5
CAT GFDK
TYPE ADF
SDIR adf222_r061

DEPN PT28HPC PDK_r0.6.1

FILE PT28HPCADF_r0.6.1.tgz

END
```

两个必须知道的“生成器缺陷”：

- 1 DEPN 写成了两字段形式 (DEPN <SKU>)，而这正是 §5.3 里“等于没写”的形式 → 由 tlp_pack 产出的依赖，安装时不会被检查。
- 2 CSV 的 VERSION（第 8 列）与 ORIG 都不会被写进 .tlp（ORIG 恒为 _）。所以版本号信息只能靠 SKU 命名本身承载，务必用 §附录 C 的命名规范。

修补办法（推荐，改 1 处 awk，已实测有效）：把 tlp_pack.awk 第 141 行的
print "DEPN\t"kit_depend >> kit_tlp
改成
print "DEPN\tKIT=\t"kit_depend >> kit_tlp
——注意 KIT= 后面那个 \t 不能省：安装端读的是第 3 个字段，写成 DEPN
KIT=<SKU>（等号后无空白）会让 \$3 为空，导致该包永远报依赖失败。详见 §11-4 与附录 C.4。

6.5 .bundle 文件

tlp_pack 每处理一个 CSV 会先 echo -n > <csv名>.bundle 清空，再把成功打包的 SKU 按序追加。它就是 §8.3 的 --bundleList 输入，也就是“这次发布的完整集合”。

第 7 章 tlp_import : 导入包清单到 dataSheets

7.1 用法

```
tlp_import [-v|--verbose] [-i|--info] [--log <file>]
           [--packageCfgDir <放.tlp的目录>] [--packageSrcDir <放.tgz的目录>]
           [--dataSheetDir <输出catalog>]
           [ <file.tlp> | <含.tlp的目录> | <SKU名> ... ]
```

行为分支 (tlp_import.csh 第 29–50 行) :

- 传目录 → 展开为该目录下 *.tlp ;
- 传文件 → 直接用 ;
- 传裸 SKU 名 → 到 TECHLIB_CFGS 下通配 <SKU>.tlp ;
- 什么都不传 → ls -1 \$TECHLIB_CFGS/*.tlp 全量导入 (目录里没有 .tlp 时报 csh 原生 ls: No match. 并静默结束) 。

7.2 它到底做了什么 (四件事)

- 1 校验载荷存在性 : 对每个 FILE 在 TECHLIB_PKGS 的冒号路径里找 tgz ; 对每个 REQU 在 TECHLIB_CFGS 里找 <x>.tlp。缺任何一个 → 整包判 error , 不产出 .dts。
- 2 拷贝并归类 : cp <x>.tlp → \$TECHLIB_DOCS/NODE/MVER/CATG/TYPE/SDIR/x.dts (内容原样, 仅改后缀) 。
- 3 登记索引 : 向 \$TECHLIB_DOCS/.tlp_package_info.csv 追加一行
NODE MVER CATG TYPE<TAB>SKU SDIR NAME ORIG (实测格式, 注意空格+Tab 混用, 用 awk -F'[\\t]+' 解析比较稳) 。
- 4 幂等与变更探测 : 目标 .dts 已存在 → WARNING: Skip import ... (already exist) ; 若内容与源 .tlp 有差异再补 WARNING: Kit '...' in the DK_RELN has been modified , 并计入 modified 计数。

7.3 实测输出

```
[tlp_import]: BEGIN
-----
[1]: Reading 'configs/GPIO_lib222_e.0.6.1.tlp' ...
    : Package type - FULL
    : Package file - packages/GPIO_lib222_e.0.6.1.tgz
    : Creating 'T28HPC/Op5/HIP/GPIO/ip222_gpio_r061/GPIO_lib222_e.0.6.1.dts' (GPIO_lib222_e.0.6.1)
    ...
-----
[tlp_import]: Total 10/10 tlp data sheets are created.
-----
```

7.4 导入报错样例 (载荷缺失 / 基线缺失)

自造一个坏包 errs2/BADKIT_missing.tlp (引用不存在的基线与 tgz) , 实测 :

```
    : Package type - FULL
    : Install base - NO_SUCH_BASE_KIT can not be found in package source.
    : Pacakge file - NO_SUCH_FILE.tgz can not be found in package source.
```

```
ERROR: Kit 'BADKIT_missing' has 2 missing package files
-----
[tlp_import]: Total 0/1 tlp data sheets are created.
[tlp_import]: Total 1/1 tlp files have missing package files. (error)
-----
```

→ `tlp_import` 因此是很好的“发布前 QA 关卡”：只要 import 报错，说明这批包不可能被正确安装。建议 CI 里判定：
`tlp_import` 输出里出现 (error) 就 fail（注意退出码恒为 0，见 §11-8）。

7.5 分类前缀 `--selectByCategory`

`-s/--selectByCategory NODE/MVER/CATG/TYPE` 只是设置 `kit_category` 变量并在 `-i` 时回声一行 `INFO: --selectByCategory ...`。它对 import/install 的实际筛选逻辑没有任何影响（README 描述的“按类别列出并预选”未实现，见 §11-7）。`run/*/Makefile` 里带着它只是为了让日志好看 + 让交互菜单的提示行有个默认值显示。

第 8 章 `tlp_install`：安装工艺库（三种模式）

`tlp_install` 是全工具的核心。它按输入自动选择在批处理模式还是交互菜单模式下运行：

给了包名/清单 ——► 批处理：逐个安装（无 tty 也能跑，适合脚本/CI）
什么都没给 ——► 交互菜单：从 `TECHLIB_DOCS` 里选（适合人肉操作）

8.1 用法

```
tlp_install [--log <f>] [-v] [-i]
  [--targetLibDir <techLib>] [--dataSheetDir <catalog>]
  [--packageSrcDir <放tgz>] [--packageCfgDir <放tlp>]
  [--bundleFile <清单>] [--selectByCategory <NODE/MVER/...>]
  [ <SKU> | <path/x.tlp> | <path/x.dts> ... ]
```

包名解析顺序（`tlp_install.csh` 54–63 行，找不到即 `exit 1` 终止整批）：

- 1 就当路径/文件存在吗 → 用之（`.dts` 也可以，交互模式选出来的就是 `.dts`）
- 2 `$TECHLIB_CFGS/<名字>` 存在吗 → 用之
- 3 `$TECHLIB_CFGS/<名字>.tlp` 存在吗 → 用之
- 4 否则 `ERROR: Can not find TLP config '<名字>' in '<TECHLIB_CFGS>'`.

这意味着批处理模式的检索目录是 `TECHLIB_CFGS`（`.tlp`），不是 `TECHLIB_DOCS`（`.dts`）。想按 `catalog` 装就得传 `.dts` 的完整路径。

8.2 模式一：按包名/路径直装

```
tlp_install --packageCfgDir configs --packageSrcDir packages \
  --targetLibDir techLib PT28HPCPDK_r0.6.1
```

实测：

```
[1]: Checking tlp package - PT28HPCPDK_r0.6.1 ...
[1]: Reading 'configs/PT28HPCPDK_r0.6.1.tlp' ...
    : Package file - packages/PT28HPCPDK_r0.6.1.tgz
```

```
INFO: Unpacking file 'packages/PT28HPCPDK_r0.6.1.tgz' ...
pdk222_r061/
pdk222_r061/README
```

```
[t1p_install]: Total 1/1 kits are installed.
```

给多个名字 = 一次装多个（按给定顺序）。安装完 6 个库后 techLib 的样子：

```
techLib/
├── .t1p_install/                ← 已安装清单（符号链接）
│   ├── PT28HPCPDK_r0.6.1.t1p -> ../T28HPC/Op5/FDK/PDK/pdk222_r061/PT28HPCPDK_r0.6.1.dts
│   └── ... (6 个)
├── .t1p_install.summary        ← 安装流水账
├── T28HPC/Op5/
│   ├── .t1p_packages           ← 本节点已装 SKU 列表（= bundle 清单来源）
│   ├── FDK/{ADF/adf222_r061, CTK/ctk222_r061, PDK/pdk222_r061}
│   ├── FIP/{MEMORY/mem222_2prf_r061, STDCELL/lib222_6t_base_e100, STDCELL/lib222_7t_base_e20}
│   └── HIP/GPIO/ip222_gpio_r061
```

对已装过的包再次执行 → WARNING: Skip install 'X' (already installed)，不覆盖、不报错，天然幂等。

8.3 模式二：--bundleFile / --bundleList（复现安装，最常用）

清单文件就是一份每行一个 SKU、顺序即安装顺序的纯文本：

```
PT28HPCPDK_r0.6.1
PT28HPCCTK_r0.6.1
PT28HPCADF_r0.6.1
STDCELL_lib222_6t_base_e.1.0
MEMORY_lib222_2PRF_r.0.6.1
GPIO_lib222_e.0.6.1

# 从现网 techLib 抽出清单
cp techLib/T28HPC/Op5/.t1p_packages myBundle.txt

# 在新机器/新目录上一比一复现
t1p_install --targetLibDir ../projA_techLib --packageCfgDir configs --packageSrcDir packages \
    --bundleList myBundle.txt
```

实测：6 行清单 → Total 8/8/Total 6/6 kits are installed.，且与原始 techLib 逐文件一致：

```
$ diff -r -x '.t1p_install*' -x '.t1p_packages' bundleLib techLib && echo IDENTICAL
IDENTICAL: bundleLib == techLib (kit content reproduced from .t1p_packages)
```

若不加 -x，make diff 一定会报差异——差异只出现在 .t1p_install.summary（含时间戳、用户名、当时的源路径）。这是正常噪声，不是安装错误。（§11-12）

--bundleFile 与 --bundleList 是同一个选项的两个别名（-b）。README

所说“中途失败可修好后用同一份 bundle 续装”成立：已装的会被 Skip install (already installed)，失败的会重来。

8.4 模式三：交互菜单

```
t1p_install --dataSheetDir dataSheets --targetLibDir techLib --packageSrcDir packages
```

第一层问类别（可反复回车直到目录存在）：

```
INFO: TECHLIB_DOCS = dataSheets
INFO: Please specify Kit Category :
└── T28HPC
```

```

└── Op5
    ├── FDK / ├── FIP / └── HIP
INPUT: Category = () ? T28HPC/Op5/FDK

```

第二层列表选包（每选一次就立刻装一次，然后回列表；#、SKU、TYPE、TOPDIR 四列）：

INPUT: Select ? 3 2 1 ← 一行可给多个序号，按给定顺序装

标记含义（实测）：行首 * = 已经装过（默认也列出，供参考），无 * = 未装。

输入	作用
1 3 5	依次安装第 1、3、5 项
0	回上一层重选类别
h / a	隐藏 / 显示“已安装（带 *）”项
t	tree -n \$TECHLIB_DOCS/<category> less 看 catalog
q	退出（直接 exit 0，不打印 END 时间戳）
非法字符 / 超范围	ERROR: Invalid selection : ... / ERROR: selection over the range : (1~N)

依赖不满足时的现场表现（交互菜单里先选 ADF 再选 PDK）：第 11 章给出正确装序。实测：ERROR: required kit dir '...' has not been installed yet. + ERROR: Skip install 'X' (dependency fail)，菜单自动重绘让你先装依赖。

8.5 每个包安装时到底执行了什么（/usr/bin/gawk -f tlp_install.awk 的 ENDFILE 段）

- 1) 读 .tlp/.dts → 取 NODE/MVER/CATG/TYPE/SDIR/FILE...
- 2) 找 .tgz（在冒号分隔的 TECHLIB_PKGS 里逐个试）；缺 → pkgs_file_missing++
- 3) 查 <SKU>.dts 标记（按 §2.4 表决定 跳过 / 冲突 / 放行）
- 4) 检查 DEPN（只有 §5.3 表里 ①②③ 三种写法有意义；KIT= 形式必须在等号后留空白）
- 5) mkdir -p ROOT/NODE/MVER/CATG/TYPE/SDIR
- 6) cp <源文件> ROOT/.../SDIR/<SKU>.dts ← “已安装”标记
- 7) ln -fs ../<类别>/<SDIR>/<SKU>.dts ROOT/.tlp_install/<SKU>.tlp ← 清单 + 依赖判据
- 8) echo "<时间> <用户> % tlp_install <SKU> \t; <源路径>" >> ROOT/.tlp_install.summary
- 9) 对 REQU 里的基线包：system("tlp_install <CFG>/<base>.tlp") 递归装（递归依赖 PATH 上有 tlp_install）
- 10) 依次 gunzip -c <tgz> | (cd ROOT/.../CATG/TYPE; tar xvf -)
- 11) echo <SKU> >> ROOT/NODE/MVER/.tlp_packages

8.6 tgz 结构约定（★最容易搞错）

解包命令的目标目录是 TECHLIB_ROOT/NODE/MVER/CATG/TYPE（不含 SDIR）。因此：

.tgz 内部必须自带一层等于 SDIR 的顶层目录，否则会解成散文件堆在 TYPE/ 下。

实测（自带样例即符合）：

```

$ tar tzf packages/PT28HPCPDK_r0.6.1.tgz
pdk222_r061/
pdk222_r061/README      ← SDIR=pdk222_r061，一致
$ tar tzf packages/PT28HPCPDK_r1.0hf7.tgz
pdk222_r10HF7/README.HF7 ← 但 configs/PT28HPCPDK_r1.0hf7.tlp 写的是 SDIR pdk222_r101 不一致

```

正确打包语句：

(cd /path/to/父目录 && tar -czf OUT/pdk222_r10HF7.tgz pdk222_r10HF7)

其他载荷规则：

- 必须是 gzip（代码是 `gunzip -c x | tar x`），裸 `.tar` 或 `zip` 不行；扩展名只是约定，`.tgz/.tar.gz` 都能被找到（查找是按 `FILE` 字段整串拼路径）。
- 不支持绝对路径/.. 逃逸（照原样 `tar x`，没做任何清洗）。安全提示：不要对不受信任来源的 `tgz` 执行安装，否则可写入 `TECHLIB_ROOT` 之外的路径。

第 9 章 tlp_check 与安装状态追溯文件

9.1 tlp_check：目前是空壳

`csh/tlp_check.csh` 只有 30 行：打印 banner → 记 `--log` → 跑一次选项解析 → 写 `END` 时间戳。不接受任何校验逻辑，任何输入都等于“什么都不做、返回 0”，并且它的 `log_file` 被硬编码成 `tlp_install.log`（会和真实安装日志互相覆盖/轮转）。

更糟的是第 1 行 shebang 写坏了：

```
#!/bin/csh -f set verbose=1      ← “set verbose=1” 被当成了第三个启动参数
```

实测：通过 `bin/tlp_check` 调用时，它 0 字节输出、不改任何日志、退出码 0，连 banner 都不打印——内核把 `-f set verbose=1` 整串交给 `csh` 后脚本根本没正常执行。只有显式 `csh -f csh/tlp_check.csh ...` 才会走到那套（毫无校验的）流程，并同时把 `tlp_install.log` 截断成它自己的两行。

README 里 `tlp_check <installed_kit_topdir> / --bundleFile / --techNode` 三种用法均未实现。

→ 把它当“完全静默的空命令”看待。校验请用 §9.3 的手写脚本。若要修：把首行改成 `#!/bin/csh -f`，并 `set log_file=tlp_check.log`。

9.2 四类状态/追溯产物

文件	由谁写	内容	用途
<code>techLib/.tlp_install/<SKU>.tlp</code>	install 第 7 步	指向库内 <code>.dts</code> 的符号链接	“已安装”权威集合； <code>DEPN KIT= <SKU></code> 形式的判据；一条 <code>ls</code> 就能列全
<code><CATG>/<TYPE>/<SDIR>/<SKU>.dts</code>	install 第 6 步	<code>.tlp</code> 的原样拷贝，落在库里	让人在 <code>techLib</code> 内部就能看到该库的包定义； <code>already installed</code> 判据
<code>techLib/.tlp_install.summary</code>	install 第 8 步	时间 用户 % <code>tlp_install SKU <TAB></code> ;源路径	审计流水（谁在什么时候装了什么）
<code>techLib/<NODE>/<MVER>/<SDIR>.tlp_packages</code>	install 第 11 步	每行一个 <code>SKU</code>	该节点已装清单，直接当 <code>--bundleList</code> 用
<code>dataSheets/.tlp_package_info.csv</code>	import 第 3 步	<code>NODE MVER CATG TYPE SKU SDIR NAME ORIG</code>	全量可用库索引（跨节点）

文件	由谁写	内容	用途
tlp_import.log / tlp_install.log / tlp_pack.log / tlp_check.log	各命令	完整过程输出（含 ANSI 色码）	故障主入口；自动轮转 .1 .2 ...

用它们做“装了什么”的答案（推荐日常三条指令）：

```
ls techLib/.tlp_install/ | 's/\.tlp$/' # 已装 SKU（权威）
cat techLib/T28HPC/Op5/.tlp_packages # 本节点已装清单（可复现）
cat techLib/.tlp_install.summary      # 安装历史（含时间/人）
```

9.3 自制的“真·check”（替代 tlp_check，已实测）

```
#!/bin/bash
# tlp_verify.sh —— 用法: tlp_verify.sh <techLibDir> <tgz源目录>
# 逐一核对 .tlp_install 清单里的每个 SKU：标记链接、库目录、载荷 tgz 是否齐全
ROOT=$1; PKGS=${2:-$1}
res=$(
for I in "$ROOT"/.tlp_install/*.tlp; do
    sku=$(basename "$I" .tlp)
    dts=$(readlink -f "$I")
    [[ -f "$dts" ]] || { echo "BAD $sku: .dts 丢失(库目录被手删)"; continue; }
    sdir=$(awk '/^SDIR[:blank:]]/{print $2; exit}' "$dts")
    [[ -d "$(dirname "$dts")/$sdir" ]] || { echo "BAD $sku: 缺目录 $sdir"; continue; }
    while read -r f; do
        [[ -f "$PKGS/$f" ]] || { echo "BAD $sku: 缺载荷 $f"; }
    done < <(awk '/^FILE[:blank:]]/{print $2}' "$dts")
    echo "OK $sku"
done | sort)
[[ -n "$res" ]] && echo "$res" || echo "(没有任何已装包)"
echo "---- $ROOT/.tlp_install 共 $(ls "$ROOT"/.tlp_install 2>/dev/null | wc -l) 条记录"
echo "$res" | grep -q '^BAD' && { echo "RESULT: 校验失败"; exit 1; }
echo "RESULT: 全部通过"
```

实测：正常库 → 6 个 OK + RESULT: 全部通过（rc=0）；手删 techLib/T28HPC/Op5/HIP 整个目录、再删一个 tgz → 准确报出

BAD GPIO…: .dts 丢失、BAD PT28HPCCTK…: 缺载荷 PT28HPCCTK_r0.6.1.tgz，rc=1。

（★细节：结果必须先收进变量再 grep。若写成 for … done | sort 后直接用循环里的标志位退出，循环在管道子 shell 里执行，bad=1 传不出来，rc 恒为 0——这是写这类校验脚本最常见的坑。）

9.4 事实上的“卸载”

工具没有 uninstall。安全的手工卸载（以一个 SKU 为例）：

```
set SKU = PT28HPCADF_r0.6.1
# 1) 先查有没有别的包依赖它（在 configs 里 grep）
grep -l "$SKU" configs/*.tlp
# 2) 删库目录 + 三个状态位
rm -rf techLib/T28HPC/Op5/FDK/ADF/adf222_r061
rm -f techLib/.tlp_install/$SKU.tlp
sed -i "/^$SKU$/d" techLib/T28HPC/Op5/.tlp_packages
# 3) 从 bundle 清单里也去掉它，否则复现安装会把它装回来
```

不要用 `rm -rf techLib` 之外的通配删除去“清理”。`techLib/` 就是设计工程师正在用的库，误删等于毁项目工作区。建议先 `cp -a techLib techLib.bak.$SKU` 再动。

第 10 章 完整实战：从零搭一套 EDA 工艺库

本章的命令与输出全部为实际执行结果（Linux + csh + gawk 5.2.1）。分两条线：A 发布侧（有裸 design kit 目录，要发包）、B 使用侧（拿到 `.tlp+.tgz`，要建 techLib）。

10.0 前置

```
git clone https://gitee.com/icdop/tlp.git tlm && cd tlm && make install
setenv TLP_HOME `pwd`          # csh 下取当前目录绝对路径
set path = ($TLP_HOME/bin $path)
which tlp_import tlp_install    # 确认能解析到
```

10.1 A线·发布：从 CSV 打包 (run/00_pack)

```
cd $TLP_HOME/run/00_pack
make help
make pack          # → configs/*.tlp + packages/*.tgz + designkit_*.bundle
```

实测（节选）：

```
[tlp_pack]: Processing TechLib Package file 'designkit_r061.csv' ...
[1]: Packing Kit 'PT28HPCPDK_r0.6.1' ...
    Create package file 'PT28HPCPDK_r0.6.1.tgz' ..
[4]: Packing Kit 'STDCELL_lib222_6t_base_e.1.0' ...
    Copy designkit to kit directory 'lib222_6t_base_e100' ... ← LOCATION basename ≠ SDIR, 走归一化分支
    Create package file 'STDCELL_lib222_6t_base_e.1.0.tgz' ..
[tlp_pack]: Total 6/6 tlp packages are created.
[tlp_pack]: Total 3/3 tlp packages are created. ← designkit_r101.csv
```

产物：`configs/` 9 个 `.tlp`、`packages/` 9 个 `.tgz`、`designkit_r061.bundle`、`designkit_r101.bundle`。

hotfix 链同理：

```
make -B patch      # 处理 patch_pdk222_r101.csv → 8 个包 + patch_pdk222_r101.bundle
```

这里必须用 `make -B`：`run/00_pack/` 下存在名为 `patch/` 的目录，与 Makefile 目标 `patch` 同名，直接 `make patch` 会得到 `make: 'patch' is up to date.` 而什么都不做。（§11-10）

10.2 B线·导入 catalog

```
cd $TLP_HOME/run/01_case
make env
make import
```

`make env`（先看配置对不对，很值）：

```
=====
TECHLIB_ROOT = techLib
TECHLIB_CFGS = configs
TECHLIB_PKGS = packages
TECHLIB_DOCS = dataSheets
=====
```


make import → Total 10/10 tlp data sheets are created. , 随后 tree -a dataSheets :

```
dataSheets/
├── .tlp_package_info.csv
├── T28HPC/Op5/
│   ├── FDK/{ADF/adf222_r061, CTK/ctk222_r061, PDK/{pdk222_r061, pdk222_r101, pdk222_r10HF4}}
│   ├── FIP/{MEMORY/mem222_2prf_r061, STDCELL/{lib222_6t_base_e100, lib222_7t_base_e20}}
│   ├── HIP/GPIO/ip222_gpio_r061
│   └── PDK/pdk222_r101      ← ★异常：分类里少了 FDK 一层
```

那个 T28HPC/Op5/PDK/... 就是自带样例的活教材：configs/PT28HPCPDK_r1.0hf7.tlp 用了老关键字 GROUP FDK，CATG 缺失 → 归类路径变成 T28HPC/Op5//PDK/（空段）。.tlp_package_info.csv 里同样留下 T28HPC Op5 PDK ... 的双空格。⇒

凡按类别检索/统计的脚本都会漏掉这个包。（§11-2）

再看“缺载荷”这一 QA 关卡（自造坏包）：

```
ERROR: Kit 'BADKIT_missing' has 2 missing pacakge files
[tlp_import]: Total 0/1 tlp data sheets are created.
[tlp_import]: Total 1/1 tlp files have missing package files. (error)
```

10.3 交互安装（人肉建 techLib）

```
make install      # 等价于 tlp_install --info --targetLibDir techLib -- ... （不给包名）
```

装序必须“先基后派”。本项目里 PDK → CTK/ADF、PDK → STDCELL/MEMORY/GPIO：

```
INPUT: Category = () ? T28HPC/Op5/FDK
INPUT: Select ? 3 2 1      ← 先 PDK(r0.6.1)，再 CTK，最后 ADF
INPUT: Category = () ? T28HPC/Op5
INPUT: Select ? 7 8 9 10  ← 其余 FIP/HIP
INPUT: Select ? q
```

若故意把 ADF 先装，实测输出（这就是文档里说的“依赖检查”）：

```
[1]: Reading 'dataSheets/.../ADF/adf222_r061/PT28HPCADF_r0.6.1.dts' ...
ERROR: required kit dir 'T28HPC/Op5/FDK/PDK/pdk222_r061' has not been installed yet.
ERROR: Skip install 'PT28HPCADF_r0.6.1' (dependency fail)
```

但这条保护只在 .tlp 写了 §5.3 中 ①②③ 三种有效形式时才存在。仓库自带 configs/*.tlp 用的是两字段 DEPN

PT28HPCPDK_r0.6.1，实测直接装成功、完全不报依赖错（§11-4）。上面这段报错来自旧 REQUIRE ... TOPDIR 语义路径下的样例数据。

结论：想让依赖真正生效，写 DEPN KIT <NODE/MVER/CATG/TYPE> TOPDIR <topdir>（等号式务必在 KIT= 后留空白）。

安装后：

```
$ cat techLib/T28HPC/Op5/.tlp_packages
PT28HPCADF_r0.6.1
PT28HPCPDK_r0.6.1
PT28HPCCTK_r0.6.1
STDCELL_lib222_6t_base_e.1.0
STDCELL_lib222_7t_base_e.2.0
GPIO_lib222_e.0.6.1
```

（顺序即安装顺序，与菜单里点的顺序一致。想让 bundle“先基线后派生”，就得按那个顺序点。）

10.4 hotfix 链实战（PATCH 机制验证）

patch_pdk222_r101.csv 定义 r1.0.1(FULL) → hf1 → hf2 → hf3 与 r1.0HF4(FULL) → hf5 → hf6 → hf7 , 全部 SDIR=pdk222_r101 :

```
tlp_install --packageCfgDir configs --packageSrcDir packages \  
--targetLibDir hotfix --bundleList patch_pdk222_r101.bundle
```

实测 :

```
[2]: Reading 'configs/PT28HPCPDK_r1.0hf1.tlp' ...  
: Package type - PATCH  
: Directory 'hotfix/T28HPC/Op5/FDK/PDK/pdk222_r101' already exist.  
INFO: Unpacking file 'packages/PT28HPCPDK_r1.0hf1.tgz' ...  
...  
[tlp_install]: Total 8/8 kits are installed.
```

结果目录是一个“叠加了 8 层”的 PDK :

```
hotfix/T28HPC/Op5/FDK/PDK/pdk222_r101/  
├── PT28HPCPDK_r1.0.1.dts ... PT28HPCPDK_r1.0hf7.dts ← 每个已装 hotfix 一枚标记 (共 7 枚可见)  
├── README ← 基线  
└── README.HF1 ... README.HF7 ← 各 hotfix 的增量说明
```

⇒ hotfix 只需上传变化的那几个文件, .tgz 极小, 装完与“直接发一个全量 r1.0HF7”等价。唯一要求: 清单顺序正确、除首个外全 PATCH。

10.5 复现与验收

```
make bundle # cp techLib/T28HPC/Op5/.tlp_packages bundleFile.txt; 装进 bundleLib  
diff -r -x '.tlp_install*' -x '.tlp_packages' bundleLib techLib && echo IDENTICAL
```

实测即 §8.3 中的 IDENTICAL。这一步骤应当作为每次“techLib 交付/换机”的固定验收关。

10.6 日常操作速查

```
# 看这个节点装了什么  
ls techLib/.tlp_install/ | sed 's/\.tlp$/ /'  
  
# 新增一个 hotfix 到已有项目  
tlp_import --packageCfgDir configs --dataSheetDir dataSheets configs/PT28HPCPDK_r1.0hf8.tlp  
tlp_install --packageCfgDir configs --packageSrcDir packages --targetLibDir techLib PT28HPCPDK_r1.0hf8  
echo PT28HPCPDK_r1.0hf8 >> techLib/T28HPC/Op5/.tlp_packages  
  
# 新项目复刻 (一条命令)  
tlp_install --packageCfgDir $TLP_ROOT/configs --packageSrcDir $TLP_ROOT/packages \  
--targetLibDir /proj/$USER/techLib --bundleList /proj/rel/T28HPC_Op5_r1.0hf7.bundle  
  
# 查包定义 (不用出 techLib)  
more techLib/T28HPC/Op5/FDK/PDK/pdk222_r101/PT28HPCPDK_r1.0.1.dts  
  
# 清掉重来 (谨慎, 先备份)  
make clean # 删 dataSheets/techLib/bundleLib/*.log
```

第 11 章 已知限制与坑 (实测清单)

全部条目均由源码核对 + 实机运行时验证得出。先读这章再上生产。

#	现象	根因（源码位置）	规避 / 修法
1	只有 csh/tcsh 能跑	所有脚本 shebang #!/bin/csh -f, 用 setenv/\$0:t/switch/glob	只在 csh 下调用；bash/zsh 用户用附录 B.2 的 csh 包装器（别用 bash 包装，见 §11-26）
2	GROUP 写了没效、catalog 出现 空目录层（T28HPC/Op5/ /PDK/）	解析只认 /^CATG\s/ (tlp _install.awk:122、tlp_im port.awk:101)	必须写 CATG；.tlp 提交前 grep -L '^CATG' *.tlp 自查
3	MD5S / SIZE / TDIR 完全不生效	只赋值不引用 (tlp_install .awk 中 kit_md5sum/file_md5 从 不参与比较；tlp_package _dir 为死变量)	“校验包完整性”得自己做 ：装前 md5sum -c，或写进 §9.3 的 check 脚本
4	依赖检查两种“看着对”的 写法全失效：① DEPN <S KU>（两字段，自带样例 与 tlp_pack 默认产出用它 ）静默不检查；② DEPN KIT=<SKU>（等号后无空 白）则 \$2 吞掉整串、\$3 为空 → 永远报依赖失败 ，任何包都装不上	tlp_install.awk 只有三条 规则：/^DEPN\s+KIT=/ 、/^DEPN\s+DIR=/、/^ DEPN\s+KIT\s/，且一律 读 \$3	只能用 ① DEPN KIT <NO DE/MVER/CATG/TYPE> TOPDIR <topdir>（推荐， 照字面即可）② DEPN KIT=<空白><SKU> ③ DEPN DIR=<空白><路径> 。五形态实测矩阵见 §5.3
5	交互菜单里安装用的是 /usr/bin/awk，批处理用 /usr/bin/gawk	tlp_install.csh:171 vs :67	/usr/bin/awk 若是 mawk ，BEGINFILE/ENDFILE 直接语法失败。执行 In -sf /usr/bin/gawk /usr/bin/awk 或 export AWK=gawk 后改脚本
6	REQU <基线>（只一列）在 import 里“看着通过”，install 却因读到空字段而 Install base - + (missing pacakge)	tlp_import.awk 读 \$2，tlp_install.awk 读 \$3（列位差一）	实测解法：基线名写两遍 REQU <基线SKU> <基线SKU> —— 这样 import 会校验基线定义存 在、install 会自动递归补 装基线（TLM 唯一的自动 依赖补齐路径，需 tlp_install 在 PATH 上）。详见《Reference Book》§6.12
7	--selectByCategory 不预选 、不筛选，照样要手输； q 退出时不写 END 时间戳	tlp_install.csh:88 无条件 set kit_category = "\$<" ;:151 exit 0	非交互场景改走 --bundleL ist；要脚本喂交互菜单见 §8.4/附录 B

#	现象	根因（源码位置）	规避 / 修法
8	装载语义失败仍返回 0（缺载荷 / 目录冲突 / 依赖不满足 / import 有 error）	.csh 末尾统一 exit 0，awk 的计数不进 exit。只有“参数层面”的错误（包名找不到、bundleFile 不存在、TECHLIB_DOCS 缺目录）才 exit 1	CI 里判日志与 stdout：`grep -qE '(error)'
9	run/00_pack/pdk222_r10hf.csv 一跑就串行（NODE=FDK、TYPE=r1.0.1…）	该文件只有 8 列（SKU 直接跟在 PACK 后），而 tlp_pack.awk:109-118 固定要 11 列	别用这个 csv（它也不在任何 Makefile 目标里）；照 §6.2 补齐 11 列
10	make patch 报 'patch' is up to date 且啥都不干	目标名与同目录 patch/ 重名	make -B patch，或把目标改名（packpatch）
11	-i/--info 打印的 TECHLIB_CFGS 值其实是 PKGS	tlp_option.csh:141-146 echo 了 \$TECHLIB_PKGS	别当诊断依据；用 make env 或直接 echo \$TECHLIB_CFGS
12	make diff 永远报差异	差异只在 .tlp_install.summary（时间戳/用户/源路径）	diff -r -x '.tlp_install*' -x '.tlp_packages' A B
13	打包在老 GNU tar / BSD tar 上可能失败	tlp_pack.awk:158,187,191,194 用 tar -c -O -z（旧式混写）	需要 GNU tar ≥1.28；BSD/macOS 上改用 gtar 或手工两步打包
14	tlp_help start/readme/format/example 打不开	指向 \$TLP_HOME/docs/TLP_QuickStart.pdf、docs/TLP_FORMAT.md —— 实际目录是 doc/，且无 PDF；还依赖 xpdf	直接 more doc/TLP_FORMAT.md；把 csh/tlp_help.csh 里 docs/ 改 doc/
15	tlp_help update 里是 svn update/ci	项目早期用 SVN	用 git 自建流程，别碰 tlp_help update
16	.tlp_packages 会被追加，重复装同一 SKU 到同一节点时可能重复行	install 第 11 步是无条件 print >>	生成 bundle 前 sort -u；或 .tlp_install/ 目录为准
17	--log 之后的选项会被当成包名（exit 1）	tlp_header.csh 的 case "-l" 缺 breaksw → 落到 default 停解析	--log 只放第一位（§4.4）；或补 breaksw
18	递归装基线要 tlp_install 在 PATH 上	tlp_install.awk:244 system("tlp_install "...） 硬用命令名	必须 make bin 且 \$TLP_HOME/bin 在 PATH，否则递归静默失败
19	TECHLIB_PKGS 未设时，awk 退化为只在 . 找；TECHLIB_CFGS 未设则在 . 找 .tlp	两个 awk 的 BEGIN 兜底	三条命令永远显式给 -c/-p

#	现象	根因（源码位置）	规避 / 修法
20	无并发保护	只有 append 写日志/清单	同一 techLib 不要并行装；或用不同 --targetLibDir
21	techLib/ 里“已装标记”与实际文件可脱节（样例 hotfix 就出现 .dts 少一枚）	先 cp .dts 再解包，解包后不复核	每次发布后跑 §9.3 的 verify；发现缺标记就手工补 cp configs/x.tlp <库目录>/x.dts
22	tlp_check 是空壳且日志写进 tlp_install.log	tlp_check.csh:20	见 §9.1/§9.3
23	不清洗 tar 路径	tar xvf - 原样解	只解可信来源包；或在隔离目录先 tar tzf 审一遍
24	未实现的选项：TECHLIB_OPTION 里的 --test、import 读的 TLP_PACK_OPTION（其他读 TECHLIB_OPTION）	变量名不一致	import 侧显式 setenv TLP_PACK_OPTION "--verbose"
25	doc/COMMAND.md 是空模板	仅 4 行占位	以本手册附录 A 为准
27	-v/--verbose、以及 TECHLIB_OPTION 里的 --verbose/--info/--test/--skip_root 全部无效	awk 写的是 for (option in arr) if (option == "--verbose") —— in 给的是下标 1,2,...，永远不等于字符串（三个 awk 引擎同一个错）。命令行 -v 还会被 tlp_header.csh 吃掉（CMDs: 行里看不到它）	别指望 -v。要看字段就直接 more 那个 .tlp/.dts；要“演练”就自己 cp 一份目标目录先试
28	tlp_pack 的 MULTI 行不按预期工作：9 个 tgz 只写进 1 条 FILE，且行为退化“把目录重新打包”	tlp_pack.awk 的分支判 pack_type，但代码只给 tlp_package_type 赋过值 → MULTI/PATCH 两个分支是死代码（同 bug 也影响 145 行 pack_type == "PATCH"）	分卷请手写多行 FILE，别用 MULTI
29	MODE TEST 是文件级开关，对该 CSV 之后的所有行都生效（包括你只想演练一行的场景）	tlp_test_mode 在 BEGINFILE 复位，但行内无“作用域”概念	演练与正式打包分成两个 CSV
30	tlp_help env 在没 setenv 时直接 TECHLIB_ROOT: Undefined variable. 崩；tlp_help command 只打印出 2 段 Usage (tlp_check 全静默，见 §9.1)	帮助脚本假定环境变量已设；tlp_check shebang 写坏	用 make env（用例 Makefile）或自己 echo \$TECHLIB_*

#	现象	根因（源码位置）	规避 / 修法
31	.bundle 生成在当前工作目录，不是 CSV 旁边	basename FILENAME .csv 把路径削掉了	打包前先 cd 到想放 bundle 的目录，或打完再 mv
32	手工删掉库里的 <SKU>.dts 标记后，重装同一 FULL 包会报 conflict root（而非“已安装”）	“已安装”判定只看该标记文件（§2.3）	恢复：cp configs/X.tlp <库目录>/X.dts 即回到 Skip install (already installed)（实测）

11.1 一句话总结这套工具的可靠性边界

tlp_import 是可信的（它的“缺包即不产出”是个好门禁）；
 tlp_install 的装载/幂等/hotfix 链/复现是可信的，但依赖检查、md5 校验都不成立；
 tlp_pack 能用但要盯 11 列；tlp_check 不存在。
 所以：顺序（bundle 清单）就是依赖管理，人是 md5。

第 12 章 报错信息速查与排查流程

12.1 报错原文 → 含义 → 处置（源码全量收集）

报错原文（含原样拼写错误）	出处	含义	处置
ERROR: Can not find TLP config 'X' in 'DIR'.	install.csh:61	包名在 TECHLIB_CFGS 里找不到	检查拼写/给完整路径/选项是否写在位置参数后
ERROR: no write permission on 'X'.	install.awk:35	无法在目标建 .tlp_install	查 --targetLibDir 权限与父目录
ERROR: dataSheet path env(TECHLIB_DOCS) is not specified.	install.csh:71	交互模式但没 catalog	先 tlp_import 或给 --dataSheetDir
ERROR: dataSheet directory 'X' does not exist.	install.csh:76	同上，目录缺失	ls -d 确认；多为相对路径 + 换目录所致
ERROR: file not found - 'X'.	install.csh:38	--bundleFile 指向的文件不存在	用 techLib/<NODE>/<MVER>/tlp_packages 生成
ERROR: Kit directory 'X' already exist before installing full kit package. + 汇总 kits have conflict root	install.awk:218	FULL 包要装的目录已被别的 SKU 占	该包应改 PATCH；或先卸载占位包；或修正 SDIR
WARNING: Skip install 'X' (already installed)	install.awk:214	幂等，正常	无需处理

报错原文 (含原样拼写错误)	出处	含义	处置
WARNING: Base directory 'X' is missing for patch package.	install.awk:226	PATCH 但基线没装	把基线排到 bundle 清单前面
ERROR: Can not install 'X' (dependency fail) + 汇总 kits require check fail	install.awk:231	有效形式的 DEPN (或旧 REQU) 未满足	先装依赖。若同时报 Required kit " ... (名字为空) → 你写成了 DEPN KIT=<SKU> 无空白形式, 见 §11-4
: Required kit 'X' has not been installed yet. / : Required kit dir 'X' ...	install.awk:132,142	上一条的具体缺项	同上
ERROR: Can not install 'X' (missing pacakge) + 汇总 kits have missing package files	install.awk:234	FILE 列的 tgz 在 TECHLIB_PKGS 里找不到	补包 / 修 FILE 行 / 扩展 TECHLIB_PKGS 冒号路径
ERROR: Pacakge file - X can not be found in package source.	import.awk:129 / install.awk:185	同上 (原样保留作者拼写 Pacakge)	同上
: Install base - X can not be found in package source.	import.awk:116 / install.awk:171	REQU 指定的基线 .tlp 不在 TECHLIB_CFGS	补齐基线定义, 或改用 DEPN
ERROR: Kit 'X' has N missing pacakge files ; 汇总 Total 0/1 ... created	import.awk:159	该 .tlp 不会进 catalog (门禁生效)	修好重跑 import
WARNING: Skip import X (already exist)	import.awk:162	幂等	需强制刷新则手删 .dts 再 import
WARNING: Kit 'X' in the DK_RELN has been modified	import.awk:164	catalog里的旧版 ≠ 源 .tlp (不更新, 只警告)	确认哪份是权威; cp 覆盖后再 import
ERROR: Invalid selection : X / selection over the range : (1~N)	install.csh:164,166	菜单输入非法	重新输入
Is: No match.	import.csh:48	TECHLIB_CFGS 下没有 .tlp	目录/选项给错
/usr/bin/gawk: ... BEGINFILE ... syntax error	—	awk 不是 gawk	§11-5
Bad case or substitution / 各类 csh 报错	—	用了 bash/zsh 或 csh 版本过老	§11-1
'X' is up to date. 但没做事	—	Makefile 目标与同名目录冲突	make -B (§11-10)

12.2 标准排查流程（装不上时按序走）

1. 看日志，别看屏幕： `more tlp_install.log` （ANSI 色码可用 `less -R`）
2. `targetLibDir/CFGs/PKGS/DOCS` 各是什么？ `tlp_install -i`（或 `make env`）
→ 特别提醒：四个默认值都是相对路径，先 `pwd` 确认
3. 该 SKU 在 catalog 里吗？ `find dataSheets -name 'X.dts'`
不在 → 跑 `tlp_import`，看是否报 `missing package`（门禁）
4. 载荷在吗？从 `X.dts` 里 `awk '/^FILE/{print $2}'` 逐个 `test -f`
5. 依赖？ `ls techLib/.tlp_install/` ← 权威“已装”集合
缺依赖 → 先核对 `DEPN` 写法是否属于 §5.3 的有效形式；否则把依赖排到 `bundle` 清单前面
6. 冲突？看是否 `FULL` 撞了已有 `SDIR` (§2.4)
7. 结构？ `tar tzf X.tgz` 首行必须等于 `SDIR/`
8. `md5`？工具不查，请自己 `md5sum` 比对 `.tlp` 的 `MD5S`

12.3 「装完发现内容不对」的恢复

由于 `techLib` 每个库目录都留了 `.dts`（=原 `.tlp`），恢复成本很低：

```
# 从现网导出清单 → 全新目录重灌 → 与旧目录逐文件比对
cp techLib/T28HPC/Op5/.tlp_packages /tmp/recover.bundle
tlp_install --packageCfgDir configs --packageSrcDir packages \
    --targetLibDir /proj/techLib.regen --bundleList /tmp/recover.bundle
diff -r -x '.tlp_install*' -x '.tlp_packages' /proj/techLib.regen techLib
# 满意后再切换（保留旧目录做回退，不要直接 rm）
mv techLib techLib.bad.$DATE ; mv /proj/techLib.regen techLib
```

附录 A 命令与选项总表

A.1 五个命令

命令	一句话	关键选项	退出码
<code>tlp_pack</code>	CSV → <code>.tlp</code> + <code>.tgz</code> + <code>.bundle</code>	<code>--packCfgsDir</code> <code>--packDestDir</code> <code>--tempDir</code>	恒 0
<code>tlp_import</code>	<code>.tlp</code> → <code>dataSheets/*.dts</code> + 索引；缺载荷即拒	<code>--packageCfgDir</code> <code>--packageSrcDir</code> <code>--dataSheetDir</code> <code>--verbose</code> <code>--info</code>	恒 0
<code>tlp_install</code>	装库；包名 / <code>--bundleList</code> / 交互 三模式	<code>--targetLibDir</code> <code>--packageCfgDir</code> <code>--packageSrcDir</code> <code>--dataSheetDir</code> <code>--bundleFile(=--bundleList)</code> <code>--selectByCategory</code>	恒 0（仅“包名找不到”等少数情形 <code>exit 1</code> ）
<code>tlp_check</code>	未实现（仅记日志）	同 <code>install</code>	恒 0
<code>tlp_help</code>	内建帮助/分发器	<code>command env readme</code> <code>start format example run</code> <code>project update</code>	—

A.2 选项 ↔ 环境变量 ↔ 默认值（`tlp_option.csh` 全量）

短	长	设置	缺省
-v	--verbose	仅置 verbose_mode (awk 侧看 TECHLIB_OPTION)	—
-i	--info	打印 ROOT/DOCS/PKGS/CFGs	—
-l	--log <f>	日志文件 (必须第一个选项)	tlp_<cmd>.log, 自动轮转 .N
-c	--packageCfgDir	TECHLIB_CFGS	\$TECHLIB_PKGS
-p	--packageSrcDir	TECHLIB_PKGS (冒号分隔多路径)	packages
-r	--dataSheetDir	TECHLIB_DOCS	dataSheets
-t	--targetLibDir	TECHLIB_ROOT	techLib
—	--packCfgDir / --packCfgsDir	TLP_CFGS_DEST	\$TLP_PKGS_DEST
—	--packDestDir	TLP_PKGS_DEST	packages
—	--tempDir	TECHLIB_TEMP	tempLib
-s	selectByCategory	csH 内 kit_category (无实际筛选作用)	空
-b	--bundleFile / --bundleList	csH 内 bundleFile	空

注意：-h/--help 不是选项解析器的一部分，是每个命令开头手写的判断，所以 \$1 必须是 -h/--help 才触发 (tlp_install --info -h 不会出帮助)。

A.3 .tlp 关键字支持矩阵 (哪个命令认哪个词)

关键字	import	install	pack 输出	有效性
BEGIN TLP <FULL PATCH>		决定冲突策略		有效
NAME / ORIG	记录	记录	(ORIG 恒 _)	仅记录
NODE MVER CATG TYPE SDIR		拼路径		核心有效
SIZE MD5S TDIR DDIR	记录	记录	—	无效果
FILE (多行)	存在性检查	解包		有效
PATCH (作为载荷 行首)	当作 FILE	当作 FILE	—	兼容旧格式
REQU / REQUIRE	取 \$2	取 \$3 + 递归装	—	列位不一致
DEPN KIT <类别> TOPDIR <topdir>	忽略	检查	(第 147 行)	★推荐、唯一照字 面即对

关键字	import	install	pack 输出	有效性
DEPN KIT= <SKU> / DEPN DIR= <路径> (等号后有空白)	忽略	检查	—	有效但反直觉
DEPN KIT=<SKU> (等号后无空白)	忽略	恒判失败	打了 C.4 旧补丁会产这个	死路
DEPN <SKU> (两列)	忽略	忽略	生成器默认产这个	无效 (§11-4)
BASE	忽略	忽略	—	无效 (只在 README/旧 hf7 样例中)
GROUP KITNAME ORIGIN DIRNAME MD5SUM PACKAGE FILE			—	老格式, 全失效
END	截断	截断		有效

附录 B 可直接复制的 Makefile 与脚本模板

B.1 项目侧 techLib/Makefile (推荐布局 : configs/ + packages/ 分家)

```
# ---- 必须 : recipe 里用了 <(…) 与 $$$ 等 bash 特性 -----
SHELL := /bin/bash

# ---- 固定配置 (务必用绝对路径) -----
export TLP_HOME := /tools/tlm
TECH_NODE      := T28HPC/Op5
TECHLIB_ROOT   := /proj/$(USER)/techLib
TECHLIB_DOCS   := $(PWD)/dataSheets
TECHLIB_CFGS   := $(PWD)/configs
TECHLIB_PKGS   := /data/tlppkg/$(TECH_NODE)

TLP            := $(TLP_HOME)/bin
OPTIONS        := --targetLibDir $(TECHLIB_ROOT) --dataSheetDir $(TECHLIB_DOCS) \
                  --packageCfgDir $(TECHLIB_CFGS) --packageSrcDir $(TECHLIB_PKGS)
REL_BUNDLE     := /proj/rel/$(subst /,_,$(TECH_NODE)).bundle

# ---- 目标 -----
.PHONY: help env import install update check verify bundle clean
help:; @echo "  make import | make install | make update | make check | make bundle | make verify"

env:
@echo "ROOT=$(TECHLIB_ROOT)\nDOCS=$(TECHLIB_DOCS)\nCFGS=$(TECHLIB_CFGS)\nPKGS=$(TECHLIB_PKGS)"

import:
$(TLP)/tlp_import $(OPTIONS) | tee import.log
@grep -q "(error)" import.log && echo "IMPORT FAILED" && exit 1 || true

install:
$(TLP)/tlp_install $(OPTIONS) --bundleList $(REL_BUNDLE) | tee install.log
@grep -qE "\\(error\\)|can not be found" install.log && exit 1 || true

check: # 列出“catalog 里有、但本机还没装”的 SKU
```

```
@comm -13 <(ls $(TECHLIB_ROOT)/.tlp_install 2>/dev/null | sed 's/\.tlp$$$/' | sort) \
    <(find $(TECHLIB_DOCS) -name '*.dts' -printf '%f\n' | sed 's/\.dts$$$/' | sort -u)
```

bundle: # 冻结当前已装集合，供他人复现

```
@cp $(TECHLIB_ROOT)/$(TECH_NODE)/.tlp_packages $(REL_BUNDLE)
```

```
@sort -u -o $(REL_BUNDLE) $(REL_BUNDLE); @cat $(REL_BUNDLE)
```

verify:

```
@bash tools/tlp_verify.sh $(TECHLIB_ROOT) $(TECHLIB_CFGS) $(TECHLIB_PKGS)
```

clean:

```
rm -rf $(TECHLIB_DOCS) import.log install.log
```

```
@echo "注意：不删 $(TECHLIB_ROOT)。要清库请显式 rm -rf 并先备份。"
```

实测：make import → Total 10/10 tlp data sheets are created. ; make check

在未装任何包时列出全部 10 个可装 SKU。

techLib / configs / packages 的路径里不要有空格（见 §11-26）。

B.2 bash/zsh 用户的一层包装（推荐做成 csh 脚本）

为什么不用 bash 写包装：TLM 参数里只要出现空格，经 bash %q 再转交 csh 就会炸出 setenv: Too many arguments。

包装器自己也写成 csh 脚本，靠 shebang 让 bash/zsh 直接调用，并用 csh 的 \$argv:q 原样透传参数（已实测）。

```
#!/bin/csh -f
# 用法: tlp <install|import|pack|check|help> [参数...]
if (! $?TLP_HOME) then
    echo "ERROR: 先 setenv TLP_HOME <工具根目录>"
    exit 1
endif
setenv PATH "$TLP_HOME/bin:$PATH"
set cmd = tlp_$1
shift argv
$cmd $argv:q

$ chmod +x tlp; export TLP_HOME=/tools/tlm
$ ./tlp install --packageCfgDir configs --packageSrcDir packages --targetLibDir tl_bash PT28HPCDPK_r0.6.1
INFO: Unpacking file 'packages/PT28HPCDPK_r0.6.1.tgz' ...
[tlp_install]: Total 1/1 kits are installed.
```

B.3 CI 质量门禁（补上 TLM 不做的 md5 与依赖校验）

```
#!/usr/bin/env bash
# gate.sh : 入仓前检查 configs/*.tlp 与 packages/*.tgz
fail=0
for f in configs/*.tlp; do
    sku=$(basename "$f" .tlp)
    grep -q '^NAME[:blank:]' "$f" || { echo "$sku: 缺 NAME"; fail=1; }
    [[ $(awk '/^NAME[:blank:]/{print $2; exit}' "$f") == "$sku" ]] \
        || { echo "$sku: NAME 与文件名不一致"; fail=1; }
    [[ $(awk '/^CATG[:blank:]/{print $2; exit}' "$f") ]] \
        || { echo "$sku: 缺 CATG (不要写 GROUP!) "; fail=1; }
    d=$(awk '/^DEPN[:blank:]/{print $2; exit}' "$f")
    [[ -z "$d" ]] || [[ $(d#*DEPN) =~ ^[:blank:]+(KIT|DIR)[:blank:]+[:blank:]*^[:blank:]+[:blank:]+TOPDIR ]] \
        || { echo "$sku: DEPN 不是推荐形式 (请用 DEPN KIT <类别> TOPDIR <topdir>) "; }
    for t in $(awk '/^FILE[:blank:]/{print $2; exit}' "$f"); do
        [[ -f packages/$t ]] || { echo "$sku: 缺载荷 $t"; fail=1; continue; }
        top=$(tar tzf "packages/$t" | head -1); top=${top%%/*}
        [[ "$top" == "$(awk '/^SDIR[:blank:]/{print $2; exit}' "$f")" ]] \
            || { echo "$sku: tgz 顶层 $top ≠ SDIR"; fail=1; }
    done
done
```

```

want=$(awk '/^MD5S[[:blank:]]/{print $2; exit}' "$f"); [[ -z "$want" ]] && continue
got=$(md5sum "packages/$t" | cut -d' ' -f1)
[[ "$got" == "$want" ]] || { echo "$sku: $t md5 不符 ($got ≠ $want)"; fail=1; }
done
done
exit $fail

```

实测（对仓库自带 configs/ + packages/ 跑）：正确抓出 PT28HPCPDK_r1.0hf7 的缺 CATG、全族无效 DEPN、三卷 tgz 与 SDIR 不一致，且无误报。

写这类脚本时别用 [\t]：grep 的方括号里 \t 是字面字符 t 不是 Tab（awk 才解释 \t），必须用 [[:blank:]]。

B.4 依赖排序器（把 TLM 不可靠的依赖检查变成确定性顺序）

```

#!/usr/bin/env bash
# topo.sh <configs目录> <请求的SKU...> → 输出拓扑排序后的安装顺序
set -o pipefail # 让 visit 的 exit 1 能穿透管道，CI 才能判失败
cfg=$1; shift
declare -A state
visit() {
    local n=$1
    case "${state[$n]:-}" in
        done) return ;; # 已输出过，跳过
        visiting) echo "循环依赖: $n" >&2; exit 1 ;;
    esac
    state[$n]=visiting
    local f="$cfg/$n.tlp"
    [[ -f $f ]] || { echo "缺定义: $n" >&2; exit 1; }
    local d
    for d in $(awk '/^DEPN[[:blank:]]+KIT=[[:blank:]]/{print $3}' "$f"); do visit "$d"; done
    for d in $(awk '/^REQU[[:blank:]]+/{print $NF}' "$f"); do [[ -f "$cfg/$d.tlp" ]] && visit "$d"; done
    state[$n]=done; printf '%s\n' "$n"
}
for s in "$@"; do visit "$s"; done | awk '!seen[$0]++'

```

实测三种情形：

```

$ bash topo.sh configs PT28HPCADF_r0.6.1 GPIO_lib222_e.0.6.1 PT28HPCPDK_r1.0hf7
PT28HPCPDK_r0.6.1      ← 基线自动排最前
PT28HPCADF_r0.6.1
GPIO_lib222_e.0.6.1
PT28HPCPDK_r1.0hf7
$ bash topo.sh configs LOOP_A ; echo $? # 构造 A→B→A
循环依赖: LOOP_A
1
$ bash topo.sh configs NOPE ; echo $?
缺定义: NOPE
1

```

用法（配合 --bundleList）：bash topo.sh configs PT28HPCADF_r0.6.1 GPIO_lib222_e.0.6.1 > order.bundle

（注意：名称式依赖要求 DEPN KIT= 之后留有空白，正则因此写成 /^DEPN[[:blank:]]+KIT=[[:blank:]]/ 并取 \$3；目录式 DEPN KIT <类别> TOPDIR <topdir> 不参与排序，得靠 bundle 清单本身的顺序保证。）

附录 C 命名规范与团队落地建议

C.1 SKU 命名规范 (.t1p 文件名 = NAME = bundle 行)

<厂商标识><NODE><TYPE>_<版本串> —— Foundry 提供
<FIP/HIP 类别>_<lib><编号>_<版本串> —— IP/单元库
例：PT28HPCPDK_r1.0hf7 ← PDK 主版本 r1.0 + hotfix 7
STDCELL_lib222_7t_base_e.2.0
MEMORY_lib222_2PRF_r.0.6.1
GPIO_lib222_e.0.6.1

规则（因为工具不携带版本字段，版本只能长在名字里）：

- 1 大小写敏感的字符保持全仓一致：r1.OHF4 与 r1.Ohf4 在 TLM 里是两个不同的包（自带样例同时存在两者）。定一个规范并强制（建议全小写 hf）。
- 2 SDIR 用“去点化”短名（r1.OHF7 → pdk222_r1OHF7），且一经发布不可改。
- 3 一个 SKU 只对应一套 .t1p + 若干 .tgz；改内容必须升 SKU，绝不覆盖已发布包。

C.2 仓库/目录治理建议

东西	放哪	版本控制
configs/*.t1p	每人可见	进 Git（这是“发布记录”本体）
dataSheets/	可再生	不进（由 import 生成），但 .t1p_package_info.csv 建议留档
packages/*.tgz	共享盘 + 只读权限	不进 Git，进制品库；chmod a-w 防覆盖
techLib/	每人/每项目	不进，只留 .t1p_packages 与 .t1p_install.summary 到项目 metadata
*.bundle	项目基线	进 Git（这就是“这一版芯片用的工艺库组合”）

C.3 SOP 建议（三条就够）

- 1 发布 SOP：gate.sh → t1p_pack/t1p_import（要求零 error）→ md5sum -c → 冻结 SKU，包目录设只读。
- 2 装机 SOP：make import && make install（走 --bundleList）→ 跑 B.4 排好的顺序 → make verify → make bundle 冻结项目基线。
- 3 升级/hotfix SOP：新 SKU 只加不改；把新 SKU 追加到项目 bundle 末尾 → t1p_install --bundleList（幂等跳过已在的）→ diff -r -x 对比变更面 → 通知/回滚靠保留旧 techLib 目录。

C.4 值得顺手做的三个小改造（一次性，收益极大）

```
# ① 让 t1p_pack 生成的依赖真正生效：DEPN 后面必须留出第 3 个字段
# ★KIT= 之后那个 \t 绝对不能省 —— 没有它就是 "$3 为空" 的死路 (§11-4)
# （只改第 141 行；第 147 行本就输出 DEPN <类别> TOPDIR <topdir>，是有效形式，别动）
sed -i 's|"DEPN\t\t"kit_depend|"DEPN\t\tKIT=\t\t"kit_depend|" csh/t1p_pack.awk
grep -n 'DEPN' csh/t1p_pack.awk # 确认第 141 行为 "DEPN\t\tKIT=\t\t"，第 147 行未变

# ② 让退出码可用（csh 语法：grep 到 "(error)" 就 exit 1，注意 $log_file 已由 header 定义）
sed -i 's|^exit 0$|grep -q "(error)" $log_file \&\& exit 1\nexit 0|' csh/t1p_install.csh csh/t1p_import.csh
```

```
# ③ 统一 awk 实现（避免交互菜单走 mawk 而崩 BEGINFILE）
sed -i 's|/usr/bin/awk|/usr/bin/gawk|' csh/tlp_install.csh
```

改完 make install 重建软链即可（软链指向 csh/，无需重装）。

②属于行为变更（原本恒返回 0），上线前先在测试用例里确认没有“可容忍的 error”被误判为失败。

附录 D 术语表

缩写	全称	说明
TLM	TechLib Management（仓库名 tlm）	本工具包整体；命令前缀却是 tlp_
TLP	TechLib Package	一个可安装的工艺库包；也是 .tlp 定义文件与所有命令的前缀
.dts	dataSheet	.tlp 被 import 后按分类归档的副本（内容相同）
DK / design kit	Design Kit	晶圆厂设计套件（PDK/CTK/ADF…）
PDK	Process Design Kit	工艺文件主体（层规则、器件模型、DFM 等）
CTK	Common/Characterization Tool Kit	表征与流程工具包
ADF	Analog Design Format	模拟设计（PCells 等）套件
FDK / FIP / HIP	Foundry Design Kit / Foundry IP / House IP	CATG 的三个取值
STDCELL / MEMORY / GPIO / AMS	单元库 / 存储器编译器 / IO / 混合信号	TYPE 取值（老格式中 AMS 已被改为 GPIO）
NODE / MVER	工艺节点 / 模型版本	安装路径的第 1、2 层
SDIR	Source/kit directory	包内顶层目录名，安装路径第 5 层
SKU	Stock Keeping Unit	包的唯一名，即 .tlp/.dts 文件名
catalog	TECHLIB_DOCS（dataSheets/）	可安装包集合
techLib	TECHLIB_ROOT	实际被 EDA 工具引用的安装树
bundle	清单	按序 SKU 列表，可完整复现一套 techLib
hotfix / HF	增量补丁包	PATCH 类型，叠加进同一 SDIR
DCM	Design Collateral Management	2017 年旧名（dcm_import/.dcm/releaseNotes/ 皆其遗留）

手册来源与可复现性

项	值
代码	<code>gitee.com/icdop/tlp / github.com/icdop/tlm</code> ，分支 <code>master</code> ， <code>HEAD 8072710</code> （共 10 次提交）
一致性	上述两公开仓库与工作副本经 <code>diff -r</code> 逐文件比对完全一致；你给的 <code>gitee.com/icdop/tlm</code> 需鉴权（401）
规模	25 个可执行/配置文件 + <code>run/00_pack</code> 、 <code>run/01_case</code> 两个用例； <code>csh/</code> 合计约 1160 行，其中核心 <code>tlp_install.awk</code> 278 行
验证环境	Linux <code>x86_64</code> ， <code>csh 20240808</code> ， <code>GNU Awk 5.2.1</code> ， <code>tar GNU</code> ， <code>tree</code>
已实跑	<code>make install</code> (建链)、 <code>run/00_pack</code> （ <code>pack / patch / import / bundle</code> ）、 <code>run/01_case</code> （ <code>env / import / 重复 import / install 交互回放 / bundle / diff</code> ）、批量安装 3 种寻址方式、依赖失败、缺载荷失败、目录冲突、 <code>ho tfix 8</code> 连装、 <code>--log/选项顺序 6</code> 组反例
未覆盖	<code>tlp_pack</code> 的 <code>MULTI / MODE TEST</code> 分支、 <code>tlp_help project</code> （仓库内无 <code>project/</code> 目录）、 <code>Cygwin</code> 侧行为

本手册中的输出片段是真实运行结果（含彩色转义已剥离、少量时间戳/用户名略去），不是推测。若你手上的仓库版本更新，请优先复核第 5 章表格与第 11 章清单——这两处最容易被后续改动影响。

手册完